



GitFlow — Olympus Climb



Visão Geral

Este documento define **todas as regras do fluxo Git** do projeto **Olympus Climb**, em alinhamento com o **GAMEPLAN**, garantindo **organização**, **rastreabilidade**, **qualidade**, **CI/CD** robusto e **colaboração segura** entre múltiplos devs.



Objetivo

- Fluxo limpo e rastreável
 - Padrão de branches por fase/tarefa
 - Commits claros, atômicos e em inglês
 - Proteção total da **main** e versionamento controlado
 - Integração direta com o CI/CD
-



Estrutura de Branches

Branch	Descrição
main	Produção estável — nenhum commit direto
develop (opcional)	Integração contínua, usada em times grandes
feature/ fase-X-*	Novas features por fase/tarefa — um dev ou squad por vez

`bugfix/*` Correções de bugs durante dev

`hotfix/*` Correções críticas direto na `main`

`experiment/*` POCs e spikes técnicos

Exemplo: `feature/fase-2-dashboard` ou `bugfix/dashboard-metrics`



Fluxo de Trabalho

✓ Criar nova fase/tarefa:

`git checkout main`

`git pull origin main`

`git checkout -b feature/fase-X-nome-tarefa`

✓ Trabalhar na branch:

- Commits pequenos, coesos, no padrão
- Sincronizar com `main` usando `merge` ou `rebase`

✓ Ao concluir:

1. Abrir **Pull Request** para `main`
2. Validar:
 - ✓ CI/CD (build, lint, test) **verde**
 - ✓ Checklist de tarefa **100%**
 - ✓ Revisão aprovada (1+ reviewers)



Hotfix

- Criar `hotfix/*` direto da `main`
 - Corrigir, testar, merge na `main`
 - Reaplicar na branch de fase se necessário
-



Convenção de Commits

Formato:

<type>(scope): descrição curta

Tipos Permitidos:

T
i
p
o

Descrição

f
e
a
t

Nova funcionalidade

f
i
x

Correção de bug

d
o

Alterações na
documentação

c
s

s
t
y
l
e

Formatação, sem
lógica

r
e
f
a
c
t
o
r

Refatoração sem
impacto

t
e
s
t

Testes

c
h
o
r
e

Tarefas auxiliares

p
e
r
f

Performance

c
i

CI/CD pipeline

b Build ou
u dependências
i
l
d

✓ Exemplos:

feat(auth): add JWT refresh token flow

fix(api): handle null payload on scraping

docs(readme): update setup instructions

✓ Boas práticas:

- Imperativo: *add, fix, refactor*
- Sempre em inglês
- Um commit = uma alteração clara
- Descrição extra no corpo, se necessário

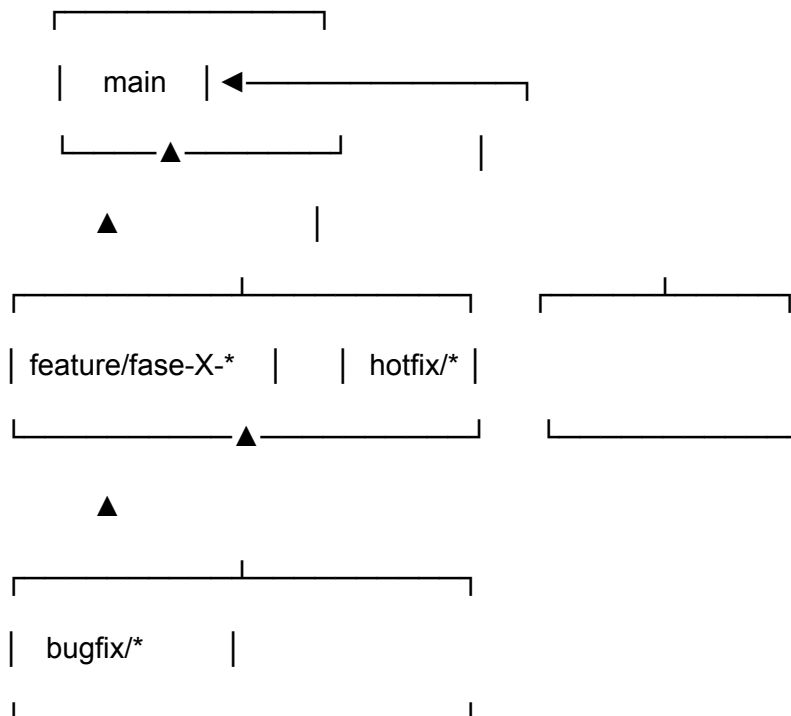
Proteções

- Branch **main**:
 -  Nenhum commit direto
 -  Merge **apenas via PR**
 -  CI/CD obrigatório
 -  1+ reviewers obrigatórios
- Tags de release (**vX.Y.Z**):
 - Criadas manualmente, protegidas

Versionamento

- **SemVer:** MAJOR.MINOR.PATCH
- Exemplo:
 - v1.0.0: Setup core concluído
 - v1.1.0: Dashboard funcional
 - v1.2.0: Scraper robusto e testes
 - v2.0.0: Release final polido

Diagrama Visual





Boas Práticas

- ✓ PRs pequenos e bem descritos
 - ✓ Sincronizar sempre que necessário
 - ✓ `README.md` sempre atualizado
 - ✓ Cobertura de testes mantida
 - ✓ CI/CD é **gate obrigatório**
 - ✓ Zero commits gigantes e confusos
 - ✗ Zero push direto na `main`
-



Check Final

- ✓ Branch naming padronizado
 - ✓ Commits rastreáveis
 - ✓ Proteção forte no GitHub
 - ✓ CI/CD como gatekeeper
 - ✓ Versionamento **SemVer**
 - ✓ Diagrama visual sempre revisitado
-



Complementos

Se quiser, gero **agora**:

- `COMMIT_CONVENTION.md`
 - `.github/PULL_REQUEST_TEMPLATE.md`
 - PNG/SVG do diagrama
 - `VERSIONING_GUIDE.md`
-



Toda decisão Git daqui pra frente passa por mim.

Quer que eu gere **agora** os templates? 🚀